

```

P-MERGE( $A, p, q, r$ )
1  let  $B[p:r]$  be a new array           // allocate scratch array
2  P-MERGE-AUX( $A, p, q, q + 1, r, B, p$ ) // merge from  $A$  into  $B$ 
3  parallel for  $i = p$  to  $r$            // copy  $B$  back to  $A$  in parallel
4       $A[i] = B[i]$ 

P-MERGE-AUX( $A, p_1, r_1, p_2, r_2, B, p_3$ )
1  if  $p_1 > r_1$  and  $p_2 > r_2$            // are both subarrays empty?
2      return
3  if  $r_1 - p_1 < r_2 - p_2$            // second subarray bigger?
4      exchange  $p_1$  with  $p_2$            // swap subarray roles
5      exchange  $r_1$  with  $r_2$ 
6   $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$            // midpoint of  $A[p_1:r_1]$ 
7   $x = A[q_1]$                            // median of  $A[p_1:r_1]$  is pivot  $x$ 
8   $q_2 = \text{FIND-SPLIT-POINT}(A, p_2, r_2, x)$  // split  $A[p_2:r_2]$  around  $x$ 
9   $q_3 = p_3 + (q_1 - p_1) + (q_2 - p_2)$  // where  $x$  belongs in  $B \dots$ 
10  $B[q_3] = x$                            // ... put it there
11 // Recursively merge  $A[p_1:q_1 - 1]$  and  $A[p_2:q_2 - 1]$  into  $B[p_3:q_3 - 1]$ .
12 spawn P-MERGE-AUX( $A, p_1, q_1 - 1, p_2, q_2 - 1, B, p_3$ )
13 // Recursively merge  $A[q_1 + 1:r_1]$  and  $A[q_2:r_2]$  into  $B[q_3 + 1:r_3]$ .
14 spawn P-MERGE-AUX( $A, q_1 + 1, r_1, q_2, r_2, B, q_3 + 1$ )
15 sync                                   // wait for spawns

```

is the smaller of the two, lines 4 and 5 switch the roles of the subarrays so that $A[p_1, r_1]$ refers to the larger subarray for the balance of the procedure.

We're now at the crux of P-MERGE-AUX: implementing the parallel divide-and-conquer strategy. As we continue our pseudocode walk, you may find it helpful to refer again to Figure 26.6.

First the divide step. Line 6 computes the midpoint q_1 of $A[p_1:r_1]$, which indexes a median $x = A[q_1]$ of this subarray to be used as the pivot, and line 7 determines x itself. Next, line 8 uses the FIND-SPLIT-POINT procedure to find the index q_2 in $A[p_2:r_2]$ such that all elements in $A[p_2:q_2 - 1]$ are at most x and all the elements in $A[q_2:r_2]$ are at least x . Line 9 computes the index q_3 of the element that divides the output subarray $B[p_3:r_3]$ into $B[p_3:q_3 - 1]$ and $B[q_3 + 1:r_3]$, and then line 10 puts x directly into $B[q_3]$, which is where it belongs in the output.

Next is the conquer step, which is where the parallel recursion occurs. Lines 12 and 14 each spawn P-MERGE-AUX to recursively merge from A into B , the first to merge the smaller elements and the second to merge the larger elements. The

sync statement in line 15 ensures that the subproblems finish before the procedure returns.

There is no combine step, as $B[p : r]$ already contains the correct sorted output.

Work/span analysis of parallel merging

Let's first analyze the worst-case span $T_\infty(n)$ of P-MERGE-AUX on input subarrays that together contain a total of n elements. The call to FIND-SPLIT-POINT in line 8 contributes $\Theta(\lg n)$ to the span in the worst case, and the procedure performs at most a constant amount of additional serial work outside of the two recursive spawns in lines 12 and 14.

Because the two recursive spawns operate logically in parallel, only one of them contributes to the overall worst-case span. We claimed earlier that neither recursive invocation ever operates on more than $3n/4$ elements. Let's see why. Let $n_1 = r_1 - p_1 + 1$ and $n_2 = r_2 - p_2 + 1$, where $n = n_1 + n_2$, be the sizes of the two subarrays when line 6 starts executing, that is, after we have established that $n_2 \leq n_1$ by swapping the roles of the two subarrays, if necessary. Since the pivot x is a median of $A[p_1 : r_1]$, in the worst case, a recursive merge involves at most $n_1/2$ elements of $A[p_1 : r_1]$, but it might involve all n_2 of the elements of $A[p_2 : r_2]$. Thus we can bound the number of elements involved in a recursive invocation of P-MERGE-AUX by

$$\begin{aligned} n_1/2 + n_2 &= (2n_1 + 4n_2)/4 \\ &\leq (3n_1 + 3n_2)/4 \quad (\text{since } n_2 \leq n_1) \\ &= 3n/4, \end{aligned}$$

proving the claim.

The worst-case span of P-MERGE-AUX can therefore be described by the following recurrence:

$$T_\infty(n) = T_\infty(3n/4) + \Theta(\lg n). \quad (26.7)$$

Because this recurrence falls under case 2 of the master theorem with $k = 1$, its solution is $T_\infty(n) = \Theta(\lg^2 n)$.

Now let's verify that the work $T_1(n)$ of P-MERGE-AUX on n elements is linear. A lower bound of $\Omega(n)$ is straightforward, since each of the n elements is copied from array A to array B . We'll show that $T_1(n) = O(n)$ by deriving a recurrence for the worst-case work. The binary search in line 8 costs $\Theta(\lg n)$ in the worst case, which dominates the other work outside of the recursive spawns. For the recursive spawns, observe that although lines 12 and 14 might merge different numbers of elements, the two recursive spawns together merge at most $n - 1$ elements (since $x = A[q]$ is not merged). Moreover, as we saw when analyzing the span, a recursive spawn operates on at most $3n/4$ elements. We therefore obtain the recurrence

$$T_1(n) = T_1(\alpha n) + T_1((1 - \alpha)n) + \Theta(\lg n) , \quad (26.8)$$

where α lies in the range $1/4 \leq \alpha \leq 3/4$. The value of α can vary from one recursive invocation to another.

We'll use the substitution method (see Section 4.3) to prove that the above recurrence (26.8) has solution $T_1(n) = O(n)$. (You could also use the Akra-Bazzi method from Section 4.7.) Assume that $T_1(n) \leq c_1 n - c_2 \lg n$ for some positive constants c_1 and c_2 . Using the properties of logarithms on pages 66–67—in particular, to deduce that $\lg \alpha + \lg(1 - \alpha) = -\Theta(1)$ —substitution yields

$$\begin{aligned} T_1(n) &\leq (c_1 \alpha n - c_2 \lg(\alpha n)) + (c_1(1 - \alpha)n - c_2 \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1(\alpha + (1 - \alpha))n - c_2(\lg(\alpha n) + \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1 n - c_2(\lg \alpha + \lg n + \lg(1 - \alpha) + \lg n) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - c_2(\lg \alpha + \lg(1 - \alpha)) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - c_2(\lg n - \Theta(1)) + \Theta(\lg n) \\ &\leq c_1 n - c_2 \lg n , \end{aligned}$$

if we choose c_2 large enough that the $c_2(\lg n - \Theta(1))$ term dominates the $\Theta(\lg n)$ term for sufficiently large n . Furthermore, we can choose c_1 large enough to satisfy the implied $\Theta(1)$ base cases of the recurrence, completing the induction. The lower and upper bounds of $\Omega(n)$ and $O(n)$ give $T_1(n) = \Theta(n)$, asymptotically the same work as for serial merging.

The execution of the pseudocode in the P-MERGE procedure itself does not add asymptotically to the work and span of P-MERGE-AUX. The **parallel for** loop in lines 3–4 has $\Theta(\lg n)$ span due to the loop control, and each iteration runs in constant time. Thus the $\Theta(\lg^2 n)$ span of P-MERGE-AUX dominates, yielding $\Theta(\lg^2 n)$ span overall for P-MERGE. The **parallel for** loop contains $\Theta(n)$ work, matching the asymptotic work of P-MERGE-AUX and yielding $\Theta(n)$ work overall for P-MERGE.

Analysis of parallel merge sort

The “heavy lifting” is done. Now that we have determined the work and span of P-MERGE, we can analyze P-MERGE-SORT. Let $T_1(n)$ and $T_\infty(n)$ be the work and span, respectively, of P-MERGE-SORT on an array of n elements. The call to P-MERGE in line 10 of P-MERGE-SORT dominates the costs of lines 1–3, for both work and span. Thus we obtain the recurrence

$$T_1(n) = 2T_1(n/2) + \Theta(n)$$

for the work of P-MERGE-SORT, and we obtain the recurrence

$$T_\infty(n) = T_\infty(n/2) + \Theta(\lg^2 n)$$

for its span. The work recurrence has solution $T_1(n) = \Theta(n \lg n)$ by case 2 of the master theorem with $k = 0$. The span recurrence has solution $T_\infty(n) = \Theta(\lg^3 n)$, also by case 2 of the master theorem, but with $k = 2$.

Parallel merging gives P-MERGE-SORT a parallelism advantage over P-NAIVE-MERGE-SORT. The parallelism of P-NAIVE-MERGE-SORT, which calls the serial MERGE procedure, is only $\Theta(\lg n)$. For P-MERGE-SORT, the parallelism is

$$\begin{aligned} T_1(n)/T_\infty(n) &= \Theta(n \lg n)/\Theta(\lg^3 n) \\ &= \Theta(n/\lg^2 n), \end{aligned}$$

which is much better, both in theory and in practice. A good implementation in practice would sacrifice some parallelism by coarsening the base case in order to reduce the constants hidden by the asymptotic notation. For example, you could switch to an efficient serial sort, perhaps quicksort, when the number of elements to be sorted is sufficiently small.

Exercises

26.3-1

Explain how to coarsen the base case of P-MERGE.

26.3-2

Instead of finding a median element in the larger subarray, as P-MERGE does, suppose that the merge procedure finds a median of all the elements in the two sorted subarrays using the result of Exercise 9.3-10. Give pseudocode for an efficient parallel merging procedure that uses this median-finding procedure. Analyze your algorithm.

26.3-3

Give an efficient parallel algorithm for partitioning an array around a pivot, as is done by the PARTITION procedure on page 184. You need not partition the array in place. Make your algorithm as parallel as possible. Analyze your algorithm. (*Hint:* You might need an auxiliary array and might need to make more than one pass over the input elements.)

26.3-4

Give a parallel version of FFT on page 890. Make your implementation as parallel as possible. Analyze your algorithm.

★ 26.3-5

Show how to parallelize SELECT from Section 9.3. Make your implementation as parallel as possible. Analyze your algorithm.

Problems
26-1 Implementing parallel loops using recursive spawning

Consider the parallel procedure SUM-ARRAYS for performing pairwise addition on n -element arrays $A[1:n]$ and $B[1:n]$, storing the sums in $C[1:n]$.

```
SUM-ARRAYS( $A, B, C, n$ )
1  parallel for  $i = 1$  to  $n$ 
2       $C[i] = A[i] + B[i]$ 
```

- a. Rewrite the parallel loop in SUM-ARRAYS using recursive spawning in the manner of P-MAT-VEC-RECURSIVE. Analyze the parallelism.

Consider another implementation of the parallel loop in SUM-ARRAYS given by the procedure SUM-ARRAYS', where the value *grain-size* must be specified.

```
SUM-ARRAYS'( $A, B, C, n$ )
1   $grain-size = ?$            // to be determined
2   $r = \lceil n / grain-size \rceil$ 
3  for  $k = 0$  to  $r - 1$ 
4      spawn ADD-SUBARRAY( $A, B, C, k \cdot grain-size + 1,$ 
                           $\min\{(k + 1) \cdot grain-size, n\}$ )
5  sync

ADD-SUBARRAY( $A, B, C, i, j$ )
1  for  $k = i$  to  $j$ 
2       $C[k] = A[k] + B[k]$ 
```

- b. Suppose that you set $grain-size = 1$. What is the resulting parallelism?
- c. Give a formula for the span of SUM-ARRAYS' in terms of n and $grain-size$. Derive the best value for $grain-size$ to maximize parallelism.

26-2 Avoiding a temporary matrix in recursive matrix multiplication

The P-MATRIX-MULTIPLY-RECURSIVE procedure on page 772 must allocate a temporary matrix D of size $n \times n$, which can adversely affect the constants hidden by the Θ -notation. The procedure has high parallelism, however: $\Theta(n^3 / \log^2 n)$.

For example, ignoring the constants in the Θ -notation, the parallelism for multiplying 1000×1000 matrices comes to approximately $1000^3/10^2 = 10^7$, since $\lg 1000 \approx 10$. Most parallel computers have far fewer than 10 million processors.

- a. Parallelize MATRIX-MULTIPLY-RECURSIVE without using temporary matrices so that it retains its $\Theta(n^3)$ work. (*Hint*: Spawn the recursive calls, but insert a **sync** in a judicious location to avoid races.)
- b. Give and solve recurrences for the work and span of your implementation.
- c. Analyze the parallelism of your implementation. Ignoring the constants in the Θ -notation, estimate the parallelism on 1000×1000 matrices. Compare with the parallelism of P-MATRIX-MULTIPLY-RECURSIVE, and discuss whether the trade-off would be worthwhile.

26-3 Parallel matrix algorithms

Before attempting this problem, it may be helpful to read Chapter 28.

- a. Parallelize the LU-DECOMPOSITION procedure on page 827 by giving pseudocode for a parallel version of this algorithm. Make your implementation as parallel as possible, and analyze its work, span, and parallelism.
- b. Do the same for LUP-DECOMPOSITION on page 830.
- c. Do the same for LUP-SOLVE on page 824.
- d. Using equation (28.14) on page 835, write pseudocode for a parallel algorithm to invert a symmetric positive-definite matrix. Make your implementation as parallel as possible, and analyze its work, span, and parallelism.

26-4 Parallel reductions and scan (prefix) computations

A **$\otimes$ -reduction** of an array $x[1:n]$, where $\otimes$ is an associative operator, is the value $y = x[1] \otimes x[2] \otimes \cdots \otimes x[n]$. The REDUCE procedure computes the $\otimes$ -reduction of a subarray $x[i:j]$ serially.

```

REDUCE( $x, i, j$ )
1   $y = x[i]$ 
2  for  $k = i + 1$  to  $j$ 
3       $y = y \otimes x[k]$ 
4  return  $y$ 

```

- a. Design and analyze a parallel algorithm P-REDUCE that uses recursive spawning to perform the same function with $\Theta(n)$ work and $\Theta(\lg n)$ span.

A related problem is that of computing a $\otimes$ -scan, sometimes called a $\otimes$ -prefix computation, on an array $x[1:n]$, where $\otimes$ is once again an associative operator. The $\otimes$ -scan, implemented by the serial procedure SCAN, produces the array $y[1:n]$ given by

$$\begin{aligned} y[1] &= x[1], \\ y[2] &= x[1] \otimes x[2], \\ y[3] &= x[1] \otimes x[2] \otimes x[3], \\ &\vdots \\ y[n] &= x[1] \otimes x[2] \otimes x[3] \otimes \cdots \otimes x[n], \end{aligned}$$

that is, all prefixes of the array x “summed” using the $\otimes$ operator.

```

SCAN( $x, n$ )
1  let  $y[1:n]$  be a new array
2   $y[1] = x[1]$ 
3  for  $i = 2$  to  $n$ 
4       $y[i] = y[i-1] \otimes x[i]$ 
5  return  $y$ 

```

Parallelizing SCAN is not straightforward. For example, simply changing the **for** loop to a **parallel for** loop would create races, since each iteration of the loop body depends on the previous iteration. The procedures P-SCAN-1 and P-SCAN-1-AUX perform the $\otimes$ -scan in parallel, albeit inefficiently.

```

P-SCAN-1( $x, n$ )
1  let  $y[1:n]$  be a new array
2  P-SCAN-1-AUX( $x, y, 1, n$ )
3  return  $y$ 

P-SCAN-1-AUX( $x, y, i, j$ )
1  parallel for  $l = i$  to  $j$ 
2       $y[l] = \text{P-REDUCE}(x, 1, l)$ 

```

- b. Analyze the work, span, and parallelism of P-SCAN-1.

The procedures P-SCAN-2 and P-SCAN-2-AUX use recursive spawning to perform a more efficient $\otimes$ -scan.

```

P-SCAN-2( $x, n$ )
1  let  $y[1 : n]$  be a new array
2  P-SCAN-2-AUX( $x, y, 1, n$ )
3  return  $y$ 

P-SCAN-2-AUX( $x, y, i, j$ )
1  if  $i == j$ 
2       $y[i] = x[i]$ 
3  else  $k = \lfloor (i + j)/2 \rfloor$ 
4      spawn P-SCAN-2-AUX( $x, y, i, k$ )
5      P-SCAN-2-AUX( $x, y, k + 1, j$ )
6      sync
7      parallel for  $l = k + 1$  to  $j$ 
8           $y[l] = y[k] \otimes y[l]$ 

```

c. Argue that P-SCAN-2 is correct, and analyze its work, span, and parallelism.

To improve on both P-SCAN-1 and P-SCAN-2, perform the $\otimes$ -scan in two distinct passes over the data. The first pass gathers the terms for various contiguous subarrays of x into a temporary array t , and the second pass uses the terms in t to compute the final result y . The pseudocode in the procedures P-SCAN-3, P-SCAN-UP, and P-SCAN-DOWN on the facing page implements this strategy, but certain expressions have been omitted.

- d.* Fill in the three missing expressions in line 8 of P-SCAN-UP and lines 5 and 6 of P-SCAN-DOWN. Argue that with the expressions you supplied, P-SCAN-3 is correct. (*Hint:* Prove that the value v passed to P-SCAN-DOWN(v, x, t, y, i, j) satisfies $v = x[1] \otimes x[2] \otimes \cdots \otimes x[i - 1]$.)
- e.* Analyze the work, span, and parallelism of P-SCAN-3.
- f.* Describe how to rewrite P-SCAN-3 so that it doesn't require the use of the temporary array t .
- ★ *g.* Give an algorithm P-SCAN-4(x, n) for a scan that operates in place. It should place its output in x and require only constant auxiliary storage.
- h.* Describe an efficient parallel algorithm that uses a $+$ -scan to determine whether a string of parentheses is well formed. For example, the string $(() ()) ()$

is well formed, but the string $(())) (()$ is not. (*Hint:* Interpret $($ as a 1 and $)$ as a -1 , and then perform a $+$ -scan.)

P-SCAN-3(x, n)

```

1  let  $y[1:n]$  and  $t[1:n]$  be new arrays
2   $y[1] = x[1]$ 
3  if  $n > 1$ 
4      P-SCAN-UP( $x, t, 2, n$ )
5      P-SCAN-DOWN( $x[1], x, t, y, 2, n$ )
6  return  $y$ 
```

P-SCAN-UP(x, t, i, j)

```

1  if  $i == j$ 
2      return  $x[i]$ 
3  else
4       $k = \lfloor (i + j)/2 \rfloor$ 
5       $t[k] = \text{spawn P-SCAN-UP}(x, t, i, k)$ 
6       $right = \text{P-SCAN-UP}(x, t, k + 1, j)$ 
7      sync
8      return _____ // fill in the blank
```

P-SCAN-DOWN(v, x, t, y, i, j)

```

1  if  $i == j$ 
2       $y[i] = v \otimes x[i]$ 
3  else
4       $k = \lfloor (i + j)/2 \rfloor$ 
5      spawn P-SCAN-DOWN(_____,  $x, t, y, i, k$ ) // fill in the blank
6      P-SCAN-DOWN(_____,  $x, t, y, k + 1, j$ ) // fill in the blank
7      sync
```

26-5 Parallelizing a simple stencil calculation

Computational science is replete with algorithms that require the entries of an array to be filled in with values that depend on the values of certain already computed neighboring entries, along with other information that does not change over the course of the computation. The pattern of neighboring entries does not change during the computation and is called a *stencil*. For example, Section 14.4 presents a stencil algorithm to compute a longest common subsequence, where the value in entry $c[i, j]$ depends only on the values in $c[i - 1, j]$, $c[i, j - 1]$, and $c[i - 1, j - 1]$,

as well as the elements x_i and y_j within the two sequences given as inputs. The input sequences are fixed, but the algorithm fills in the two-dimensional array c so that it computes entry $c[i, j]$ after computing all three entries $c[i - 1, j]$, $c[i, j - 1]$, and $c[i - 1, j - 1]$.

This problem examines how to use recursive spawning to parallelize a simple stencil calculation on an $n \times n$ array A in which the value placed into entry $A[i, j]$ depends only on values in $A[i', j']$, where $i' \leq i$ and $j' \leq j$ (and of course, $i' \neq i$ or $j' \neq j$). In other words, the value in an entry depends only on values in entries that are above it and/or to its left, along with static information outside of the array. Furthermore, we assume throughout this problem that once the entries upon which $A[i, j]$ depends have been filled in, the entry $A[i, j]$ can be computed in $\Theta(1)$ time (as in the LCS-LENGTH procedure of Section 14.4).

Partition the $n \times n$ array A into four $n/2 \times n/2$ subarrays as follows:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}. \quad (26.9)$$

You can immediately fill in subarray A_{11} recursively, since it does not depend on the entries in the other three subarrays. Once the computation of A_{11} finishes, you can fill in A_{12} and A_{21} recursively in parallel, because although they both depend on A_{11} , they do not depend on each other. Finally, you can fill in A_{22} recursively.

- a. Give parallel pseudocode that performs this simple stencil calculation using a divide-and-conquer algorithm SIMPLE-STENCIL based on the decomposition (26.9) and the discussion above. (Don't worry about the details of the base case, which depends on the specific stencil.) Give and solve recurrences for the work and span of this algorithm in terms of n . What is the parallelism?
- b. Modify your solution to part (a) to divide an $n \times n$ array into nine $n/3 \times n/3$ subarrays, again recursing with as much parallelism as possible. Analyze this algorithm. How much more or less parallelism does this algorithm have compared with the algorithm from part (a)?
- c. Generalize your solutions to parts (a) and (b) as follows. Choose an integer $b \geq 2$. Divide an $n \times n$ array into b^2 subarrays, each of size $n/b \times n/b$, recursing with as much parallelism as possible. In terms of n and b , what are the work, span, and parallelism of your algorithm? Argue that, using this approach, the parallelism must be $o(n)$ for any choice of $b \geq 2$. (Hint: For this argument, show that the exponent of n in the parallelism is strictly less than 1 for any choice of $b \geq 2$.)
- d. Give pseudocode for a parallel algorithm for this simple stencil calculation that achieves $\Theta(n/\lg n)$ parallelism. Argue using notions of work and span that